

Saga Pattern vs Circuit Breaker Pattern:

Introducción

La evolución de la ingeniería de software hacia arquitecturas nativas de la nube ha redefinido fundamentalmente la forma en que los sistemas empresariales procesan y gestionan la información. En el epicentro de esta transformación se encuentra el desafío de mantener la integridad operativa y la consistencia de los datos cuando los procesos de negocio trascienden los límites de un único espacio de memoria o una única base de datos. La adopción de sistemas distribuidos introduce fallos inherentes de red, latencia impredecible y la imposibilidad teórica de garantizar simultáneamente una consistencia estricta y una disponibilidad total ante particiones de red, un axioma formalizado en el Teorema CAP y su corolario, el Teorema PACELC¹.

El presente documento de investigación proporciona un análisis exhaustivo de dos patrones de diseño arquitectónico críticos que abordan estos desafíos desde dimensiones ortogonales: el patrón Saga y el patrón Circuit Breaker. El patrón Saga se erige como una solución algorítmica para la gestión de transacciones distribuidas de larga duración, resolviendo el problema de la consistencia de datos sin depender de protocolos de bloqueo pesados que degradan el rendimiento⁴. Por su parte, el patrón Circuit Breaker aborda la tolerancia a fallos a nivel de infraestructura, previniendo el agotamiento de recursos sistémicos y los fallos en cascada derivados de la comunicación sincrónica entre servicios⁶.

El objetivo de esta investigación es desglosar la mecánica teórica, las ventajas, las limitaciones y las estrategias de implementación de ambos patrones. Se explorará cómo funcionan de manera aislada, en qué contextos arquitectónicos resultan indispensables, cómo se complementan dentro de un ecosistema unificado y qué metodologías de prueba son necesarias para demostrar su viabilidad en una Prueba de Concepto (PoC).

Microservicios y transacciones distribuidas

La transición desde sistemas monolíticos hacia arquitecturas de microservicios introduce una complejidad profunda en el manejo del estado y la atomicidad de las operaciones. En un diseño de microservicios ortodoxo, cada servicio opera como una entidad autónoma responsable de un dominio de negocio específico, encapsulando su propia lógica y, de manera crítica, su propia base de datos⁸.

Esta característica, conocida como el patrón de base de datos por servicio (Database-per-Service), garantiza el desacoplamiento, permite la escalabilidad independiente y facilita despliegues ágiles. Sin embargo, fractura la capacidad del sistema para utilizar los motores de transacciones relacionales tradicionales. Una operación de negocio típica, como el procesamiento de una compra, requiere la actualización coordinada de múltiples microservicios

(por ejemplo, validar el saldo, descontar inventario y programar la logística). Cuando todas estas operaciones residían en una única base de datos, una transacción local garantizaba el cumplimiento de las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad).

La atomicidad aseguraba un resultado de "todo o nada", mientras que los mecanismos de aislamiento (mediante bloqueos o control de concurrencia multiversión) prevenían interferencias entre transacciones concurrentes¹.

Una transacción distribuida, en marcado contraste, es aquella donde las operaciones de escritura y lectura atraviesan múltiples recursos de red y bases de datos independientes. Lograr garantías ACID en este contexto resulta extraordinariamente complejo porque no existe un árbitro centralizado capaz de confirmar instantáneamente el estado global sin incurrir en latencias de red masivas. El Teorema CAP establece que un sistema distribuido frente a una partición de red debe elegir entre consistencia y disponibilidad¹. Más aún, el Teorema PACELC extiende esta noción, indicando que incluso bajo condiciones normales de red, el arquitecto debe hacer un intercambio explícito (trade-off) entre la latencia y la consistencia de los datos³. Intentar forzar la consistencia fuerte (strong consistency) en microservicios frecuentemente conduce a sistemas frágiles y de alta latencia.

El mecanismo histórico para intentar resolver esto es el protocolo Two-Phase Commit (2PC), formalizado por Jim Gray en 1978¹¹. El 2PC utiliza un coordinador central para dirigir a múltiples participantes (bases de datos) a través de dos fases estrictas. En la Fase de Preparación, el coordinador interroga a los participantes si están listos para confirmar; los participantes validan la operación, escriben en sus registros durables (write-ahead logs) y retienen bloqueos pesados (pessimistic locks) sobre las filas afectadas, respondiendo con un voto de compromiso. En la Fase de Confirmación, si todos los votos son afirmativos, el coordinador ordena la aplicación definitiva de los cambios¹³.

Aunque el 2PC proporciona una consistencia estricta, posee vulnerabilidades arquitectónicas fatales para los microservicios distribuidos modernos. Fundamentalmente, es un protocolo bloqueante (blocking protocol). Si el coordinador sufre una falla catastrófica después de la fase de preparación pero antes de la confirmación, los participantes quedan en un estado de incertidumbre, incapaces de liberar los bloqueos sobre sus recursos locales hasta que el coordinador sea restaurado¹⁴. Esto detiene por completo cualquier otra operación que necesite acceder a dichos datos. Además, estudios empíricos demuestran que el rendimiento transaccional del 2PC se degrada severamente a medida que aumenta el número de servicios involucrados; en simulaciones con cargas altas, la latencia aumenta exponencialmente y la eficiencia cae drásticamente, haciendo que el 2PC sea inviable para entornos de alto rendimiento o escalabilidad masiva¹⁷. No se debe catalogar al 2PC como inherentemente incorrecto; es la herramienta adecuada cuando la consistencia absoluta es obligatoria legalmente y la escala es pequeña. Sin embargo, en arquitecturas nativas de la nube orientadas a la alta disponibilidad, su rigidez lo convierte en un cuello de botella sistémico.

Saga Pattern

Para reconciliar la necesidad imperativa de consistencia de datos con las demandas de alta disponibilidad y bajo acoplamiento de los microservicios, la industria ha adoptado el patrón Saga. Introducido originalmente en la literatura académica en 1987 por Hector Garcia-Molina y Kenneth Salem como una técnica para gestionar transacciones de base de datos de larga duración, el patrón ha sido readaptado magistralmente para ecosistemas distribuidos⁵.

El patrón Saga resuelve el problema fundamental de la actualización coordinada de múltiples bases de datos sin recurrir a bloqueos distribuidos. A nivel conceptual, una Saga descompone una transacción de negocio global en una secuencia algorítmica y orquestada de transacciones locales aisladas⁴. Cada transacción local dentro de la Saga se ejecuta en el contexto exclusivo de un único microservicio, utilizando las garantías ACID nativas de su propia base de datos. Una vez que esta transacción local se confirma con éxito (commit), el microservicio publica un mensaje o evento de dominio que desencadena de forma asíncrona el siguiente paso de la secuencia¹⁹.

La piedra angular teórica de este modelo es la aceptación de la Consistencia Eventual. Al renunciar a las transacciones distribuidas, el sistema asume que el estado global de los datos será transitoriamente inconsistente mientras la Saga está en vuelo. Sin embargo, el patrón garantiza matemáticamente que, al finalizar el proceso, la transacción de negocio habrá avanzado completamente hacia el éxito, o se habrá retrotraído limpiamente a través de transacciones compensatorias, logrando así la consistencia final⁴.

Para comprender su funcionamiento, es útil examinar un flujo transaccional genérico en una plataforma de comercio electrónico. La operación de negocio comienza cuando el microservicio de Órdenes recibe una solicitud. Este servicio inicia una transacción local, crea el registro del pedido en un estado intermedio (por ejemplo, PENDING) y confirma (commit) en su base de datos. Seguidamente, se emite una señal para el microservicio de Pagos. Este último inicia su propia transacción local, interactúa con la pasarela financiera, deduce los fondos de la cuenta del cliente y confirma la operación, liberando inmediatamente cualquier recurso o hilo. El éxito del pago desencadena la acción del microservicio de Inventario, que reserva físicamente las unidades solicitadas mediante otra transacción local. Finalmente, el microservicio de Envíos recibe la confirmación de inventario, programa la logística y marca su proceso como exitoso. Concluidos todos los pasos, la Saga se da por terminada y el estado global converge, actualizando la orden a APPROVED.

Durante todo este proceso, ningún microservicio mantuvo conexiones abiertas ni bases de datos bloqueadas esperando la resolución de los demás servicios, maximizando el rendimiento (throughput) global del sistema¹⁷.

Coreografía

Existen dos estrategias topológicas para gestionar el flujo de las transacciones locales dentro de una Saga: Coreografía y Orquestación. La Saga por Coreografía es un enfoque

descentralizado basado en la arquitectura dirigida por eventos (Event-Driven Architecture), donde no existe un punto central de control ni un coordinador explícito que dicte el flujo de la transacción de negocio⁴.

En este modelo, cada microservicio participante es totalmente autónomo. Su funcionamiento se basa en la reacción a eventos de dominio emitidos por otros microservicios. Cuando un participante completa exitosamente su transacción local, emite un evento a través de un bus de mensajes (Message Broker). Otros microservicios, que han suscrito un interés previo en dichos eventos, los consumen asíncronamente y ejecutan su propia lógica funcional, emitiendo posteriormente sus propios eventos resultantes⁴.

En el contexto del comercio electrónico, el flujo conceptual operaría de la siguiente manera: El *Order Service* crea la orden y publica el evento *OrderCreated*. El *Payment Service*, que actúa como consumidor de este tópico, detecta el evento, procesa el cobro y, al finalizar, actúa como productor publicando *PaymentCompleted*. El *Inventory Service*, reaccionando exclusivamente a *PaymentCompleted*, descuenta las unidades y publica *InventoryReserved*. Finalmente, el *Shipping Service* detecta la reserva, agenda el paquete y emite *ShippingScheduled*, lo cual puede ser escuchado por *Order Service* para actualizar el estado final de la orden. Ningún servicio instruye directamente a otro sobre qué hacer; simplemente informan sobre hechos consumados en sus dominios.

Este enfoque presenta ventajas significativas en términos de agilidad organizativa y desacoplamiento. Dado que los servicios no tienen dependencias explícitas de API entre sí, los equipos de desarrollo pueden iterar, escalar y desplegar sus microservicios de manera completamente independiente. La ausencia de un orquestador central elimina un potencial cuello de botella y punto único de fallo (single point of failure), ofreciendo una escalabilidad horizontal excepcional⁴. Sin embargo, las desventajas emergen dramáticamente a medida que el sistema evoluciona. La complejidad cognitiva se dispara, ya que el flujo del proceso de negocio no está documentado en ninguna línea de código centralizada, sino que existe implícitamente como la suma de múltiples suscripciones dispersas. Esto genera un alto acoplamiento a nivel de eventos y una observabilidad precaria, dificultando la detección de ciclos infinitos, el rastreo de transacciones atascadas y la comprensión integral del sistema sin herramientas avanzadas de telemetría distribuida¹⁹.

Orquestación

En contraste directo con la naturaleza emergente de la coreografía, la Saga por Orquestación introduce un componente centralizado explícito: el Orquestador de Saga. Este elemento actúa como el cerebro director de la transacción distribuida, centralizando la lógica de flujo, las ramificaciones condicionales y el manejo exhaustivo de las compensaciones⁴.

El funcionamiento se basa en un patrón de comando y control. El orquestador mantiene una máquina de estados interna que modela el ciclo de vida exacto del proceso de negocio. En

lugar de que los microservicios reaccionen pasivamente a eventos de dominio, el orquestador envía comandos asíncronos o sincrónicos de manera proactiva a cada participante, indicándole con precisión la acción a ejecutar. Tras ejecutar su transacción local, el microservicio envía un mensaje de respuesta (éxito o fracaso) de vuelta al orquestador. Basado en esta respuesta, el orquestador evalúa su máquina de estados, actualiza el progreso en su propia base de datos persistente y emite el comando para la siguiente etapa⁴.

Tomando el ejemplo previo, el *Order Service* instanciaría un proceso en el *OrderSagaOrchestrator*. Este orquestador emite un comando *ProcessPaymentCommand* dirigido al *Payment Service*. Al recibir una respuesta *PaymentSuccessResponse*, el orquestador avanza su estado y despacha un *ReserveInventoryCommand* al *Inventory Service*. Si este responde con *InventoryFailedResponse*, el orquestador es el único responsable de iniciar la ruta de fallo, enviando un *RefundPaymentCommand* al servicio de pagos. Las ventajas de la orquestación son formidables para dominios empresariales complejos. Centraliza la definición del proceso de negocio, haciendo que el flujo sea inmediatamente comprensible, fácil de auditar y sumamente sencillo de probar mediante pruebas unitarias o de integración centralizadas. La observabilidad se simplifica enormemente, ya que el estado exacto de cualquier transacción global puede consultarse interrogando la base de datos del orquestador⁴. No obstante, este enfoque conlleva desventajas críticas. El orquestador puede convertirse en un sumidero de lógica de negocio que acopla fuertemente a los servicios participantes a sus contratos de comando. Además, introduce un punto centralizado de control que, si no se diseña con un alto grado de redundancia y resiliencia de estado (*durable execution*), puede paralizar la creación de nuevas transacciones y bloquear las que están en curso ante una falla del sistema⁴.

Aspecto Analizado	Saga por Coreografía	Saga por Orquestación
Complejidad Arquitectónica	Baja en etapas iniciales, pero crece exponencialmente y se vuelve opaca a gran escala.	Alta al inicio por la necesidad de construir y mantener el motor de orquestación.
Nivel de Acoplamiento	Extremadamente bajo. Los servicios son ciegos a la existencia de otros dominios.	Moderado a alto. El orquestador debe conocer los comandos de todos los participantes.
Escalabilidad	Sobresaliente. Ausencia de cuellos de botella centralizados o bloqueos.	Muy buena, pero estrictamente limitada por la capacidad transaccional del

		orquestador.
Mantenibilidad	Desafiante. Modificar el flujo implica alterar suscripciones en múltiples servicios.	Óptima. Toda alteración de la lógica del proceso se realiza en un único componente.
Observabilidad	Fragmentada. Requiere sistemas de rastreo distribuido y correlación de logs complejos.	Centralizada y directa. El estado del proceso global se persiste en un único lugar.
Manejo de Errores	Distribuido. Cada servicio debe saber qué evento de error escuchar y cómo reaccionar.	Estricto. El orquestador central decide y despacha las rutinas de compensación.
Facilidad de Pruebas	Pobre. Demostrar el flujo completo requiere levantar todo el ecosistema de servicios.	Excelente. El orquestador puede probarse aislando dependencias mediante simuladores (mocks).
Cantidad de Componentes	Menor. Utiliza la infraestructura de mensajería existente.	Mayor. Requiere el despliegue y mantenimiento de los nodos orquestadores y su almacenamiento.
Casos de Uso Apropriados	Flujos cortos (2-4 pasos), altamente estables, desarrollados por equipos descentralizados.	Flujos largos (5+ pasos), dinámicos, que exigen alta trazabilidad y control regulatorio estricto.

Ninguno de los enfoques se consolida como un ganador universal. La decisión arquitectónica debe basarse en la topología organizacional, la longitud de la transacción y los requerimientos estrictos de observabilidad de cada dominio particular.

Compensaciones

El componente más crítico y conceptualmente exigente de una Saga es el diseño de las operaciones de compensación. En una base de datos relacional monolítica, si ocurre un error a mitad del procesamiento, el sistema emite un comando ROLLBACK. Este comando destruye inmediatamente los cambios pendientes en memoria y devuelve los datos a su estado original, asegurando que ninguna otra transacción haya presenciado las mutaciones incompletas⁴.

En el patrón Saga, este mecanismo de recuperación es técnicamente imposible. Debido a que la Saga se compone de una cadena de transacciones locales independientes, los datos de las fases iniciales ya han sido comprometidos (committed) permanentemente en sus respectivas bases de datos y, crucialmente, ya son visibles y manipulables por otras operaciones concurrentes de la plataforma. Para restaurar la coherencia del negocio tras un fallo, la Saga no realiza un rollback tradicional, sino que ejecuta transacciones compensatorias. Una transacción compensatoria es una nueva operación de negocio afirmativa cuyo propósito semántico es deshacer, mitigar o revertir los efectos de una transacción previamente confirmada⁴.

La distinción entre un rollback de base de datos y una compensación de negocio es profunda. Algunas operaciones del mundo real carecen de una función matemática inversa. Por ejemplo, si un servicio envía un correo electrónico de confirmación, es imposible "des-enviar" ese correo. La compensación semántica debe ser el envío de un nuevo correo explicando la cancelación²⁰. Para estructurar esto teóricamente, Chris Richardson clasifica los pasos de una Saga en tres categorías estrictas:

1. **Transacciones Compensables (Compensable):** Son aquellas operaciones en las fases iniciales de la Saga que pueden y deben revertirse si el flujo falla posteriormente (por ejemplo, congelar fondos en una tarjeta de crédito, que se compensa mediante un desembolso o reembolso)⁴.
2. **Transacciones Pivote (Pivot):** Representan el punto de no retorno dentro del proceso de negocio. Si la transacción pivote se completa con éxito, la Saga entra en un estado de compromiso absoluto para finalizar exitosamente. Un paso pivote puede ser la última operación reversible o la primera irrevocable⁴.
3. **Transacciones Reintentables (Retryable):** Todas las operaciones posteriores al pivote. Estas transacciones están garantizadas para tener éxito eventualmente. Si fallan por problemas de red o caídas de infraestructura, el sistema simplemente aplicará retiros exponenciales y backoff hasta que se concreten. Nunca se retrocede después del pivote⁴.

Consideremos el flujo donde se ha creado la orden y procesado el pago, pero al intentar reservar el inventario ocurre una falla irrecuperable (falta de stock físico). La transacción de inventario falla, desencadenando la fase de recuperación de la Saga. El sistema debe ahora operar en orden estrictamente inverso. Primero, ejecuta el comando para compensar el pago, instanciando una transacción local en el servicio financiero para reintegrar el saldo. Una vez confirmada esta compensación, ejecuta la compensación final en el servicio de órdenes, marcando el pedido como cancelado permanentemente.

El desafío de ingeniería más grande surge si la propia compensación falla. Si al intentar realizar el reembolso la pasarela financiera externa se encuentra caída, la Saga no puede simplemente

rendirse, ya que el sistema quedaría en una inconsistencia financiera severa. Por ende, toda transacción compensatoria debe ser diseñada para ser implacablemente reintentada y, como requisito matemático innegociable, debe ser completamente **idempotente**. La idempotencia garantiza que ejecutar la operación de reembolso una, dos o cien veces (debido a reintentos o mensajes duplicados en la red) producirá exactamente el mismo estado final sin aplicar reembolsos múltiples²⁰.

Problemas de Saga

La adopción del patrón Saga requiere que los arquitectos enfrenten frontalmente las complejidades inherentes a la programación asíncrona distribuida y a los modelos de consistencia eventual. La mitigación de estos problemas demanda estrategias algorítmicas de alto nivel sin depender puramente de frameworks.

El problema primario emana de las anomalías de aislamiento de la base de datos (Database Isolation Anomalies). En el contexto de ACID tradicional, el aislamiento previene que las transacciones simultáneas colisionen. En Saga, dado que los bloqueos se liberan tempranamente, el sistema opera por defecto en un nivel de aislamiento comparable al "Read Uncommitted", abriendo la puerta a tres vulnerabilidades severas⁴:

1. **Lecturas Sucias (Dirty Reads):** Una transacción ajena lee datos modificados por una Saga que aún no ha terminado. Si posteriormente la Saga experimenta un fallo y ejecuta compensaciones, la transacción externa operó sobre información ilusoria y falsa⁴.
2. **Pérdida de Actualizaciones (Lost Updates):** Dos Sagas concurrentes interactúan con el mismo registro. Una sobrescribe los cambios de la otra porque ignoró el estado intermedio de su predecesora, destruyendo la integridad de los datos⁴.
3. **Lecturas Borrosas (Fuzzy Reads):** Una misma Saga interroga un registro en dos pasos diferentes de su ciclo y descubre que el valor mutó drásticamente debido a la intervención de otro proceso paralelo⁴.

La estrategia general para erradicar estas anomalías sin utilizar bloqueos centralizados implica la implementación de bloqueos semánticos (Semantic Locks). Esto significa alterar el esquema de datos para reflejar estados de transición. Por ejemplo, en lugar de mutar un saldo directamente, el registro adquiere un indicador de estado como PENDING_UPDATE, instruyendo a otras transacciones a rechazar operaciones concurrentes hasta que la Saga actual reclame el éxito o la compensación definitiva.

A nivel operativo de red, la asincronía introduce el problema de los mensajes duplicados, los mensajes perdidos y los mensajes fuera de orden. Las redes distribuidas son inherentemente no confiables. La solución a la duplicación recae, nuevamente, en la Idempotencia estricta implementada mediante claves de correlación (Idempotency Keys) únicas generadas en el inicio del flujo. Para los mensajes perdidos, es forzoso el uso de repositorios persistentes de eventos y patrones de confirmación de entrega (acknowledgments). Si un mensaje llega fuera de orden (por ejemplo, recibir una petición de compensación antes de que se procese la petición original de creación), el servicio receptor debe estar programado para registrar el evento futuro o ignorar transacciones obsoletas mediante un control de concurrencia optimista (Optimistic Concurrency Control) basado en versiones⁵.

Estados del ciclo de vida

Para mitigar la opacidad operacional mencionada anteriormente, la representación formal del estado de una Saga es ineludible. Un proceso distribuido debe modelarse a través de un ciclo de vida rastreable que refleje fielmente su progreso asíncrono en cualquier instante dado:

- **Iniciada (Started):** El sistema ha aceptado el requerimiento inicial y ha comenzado a procesar el primer eslabón transaccional.
- **En ejecución (Executing):** La Saga avanza por la vía principal de éxito. Las transacciones compensables están cometiéndose escalonadamente.
- **Completada (Completed):** La barrera de la transacción pivote ha sido cruzada y las transacciones reintentables finalizaron exitosamente. El negocio ha concluido de forma coherente.
- **Fallida (Failed / Aborted):** Una validación de negocio ha rechazado el progreso en una transacción compensable. El avance se detiene, marcando el inicio de la contingencia.
- **Compensando (Compensating):** El orquestador o la coreografía ha comenzado a ejecutar la cadena de eventos de negocio en reversa, emitiendo comandos mitigadores.
- **Compensada (Compensated / Rolled Back):** Todas las transacciones compensatorias concluyeron exitosamente con confirmación del sistema. La consistencia eventual se ha consolidado regresando el equilibrio.

Rastrear estos estados, ya sea en la base de datos de un orquestador o inferidos a través de arquitecturas de agregación de eventos, permite exponer al usuario interfaces honestas, informando, por ejemplo, que una transacción se encuentra "Bajo revisión" durante una fase de compensación, en lugar de mostrar datos corruptos o contradictorios procedentes de lecturas sucias.

Circuit Breaker Pattern

Mientras que el patrón Saga orquesta majestuosamente la consistencia de los datos del negocio ante fallas lógicas, es ineficaz contra fallas catastróficas de la infraestructura subyacente. Para blindar las comunicaciones de red entre microservicios interdependientes, la arquitectura exige la implementación del patrón Circuit Breaker.

Definición y Fallos en cascada

Conceptualizado por Michael Nygard en su obra fundamental *Release It!*, el patrón Circuit Breaker actúa como un mecanismo de protección proxy ubicado entre un cliente y el servicio remoto al que invoca⁶. Su funcionamiento emula directamente a los disyuntores eléctricos que interrumpen el flujo de corriente cuando detectan sobrecargas para evitar el incendio de la instalación. En software, el patrón monitoriza incesantemente los resultados (éxitos, excepciones de red, latencias) de las peticiones salientes. Si detecta que una dependencia externa presenta niveles anormales de degradación, el circuito "se abre" o se dispara, deteniendo automáticamente todas las llamadas posteriores hacia dicho servicio⁶.

El problema cardinal que resuelve es la propagación sistémica de la saturación de recursos, comúnmente descrita mediante la teoría de colas y que conduce a fallos en cascada masivos. Considere un escenario donde el *Servicio A* depende sincrónicamente del *Servicio B*, que a su

vez consulta al *Servicio C*. Si el *Servicio C* sufre una degradación grave en su base de datos y comienza a tardar treinta segundos en procesar cada petición (o simplemente no responde hasta agotar los tiempos de espera), las consecuencias son fatales⁷.

El *Servicio B*, ignorante del problema de fondo, continúa recibiendo un volumen alto de tráfico del *Servicio A*. Para procesar cada una de estas peticiones entrantes, *B* abre un hilo (thread) o una conexión TCP dedicada y envía la petición hacia *C*. Debido a la extrema lentitud de *C*, estos hilos no se liberan. En cuestión de segundos, la piscina de hilos (thread pool) entera del *Servicio B* se satura por completo. *B* se paraliza. Consecuentemente, el *Servicio A* comienza a apilar sus propios hilos esperando a *B*, hasta que *A* también colapsa. Un problema aislado en la base de datos de *C* ha causado la inmolación total de todo el ecosistema distribuido debido al agotamiento de recursos (resource starvation)⁷.

El Circuit Breaker previene esto. Al estar integrado en el *Servicio B*, detectaría los primeros fallos y alta latencia hacia *C*. Superado un umbral estadístico, cortaría la ruta. A partir de ese momento, cualquier petición de *B* hacia *C* fallaría de inmediato a nivel de la memoria local de *B* (Fail Fast), sin intentar abrir conexiones de red ni bloquear hilos. El *Servicio B* rechaza procesar dependencias tóxicas, salvaguardando sus propios hilos y manteniendo su disponibilidad parcial para otras funciones no afectadas⁶.

Estados del Circuit Breaker

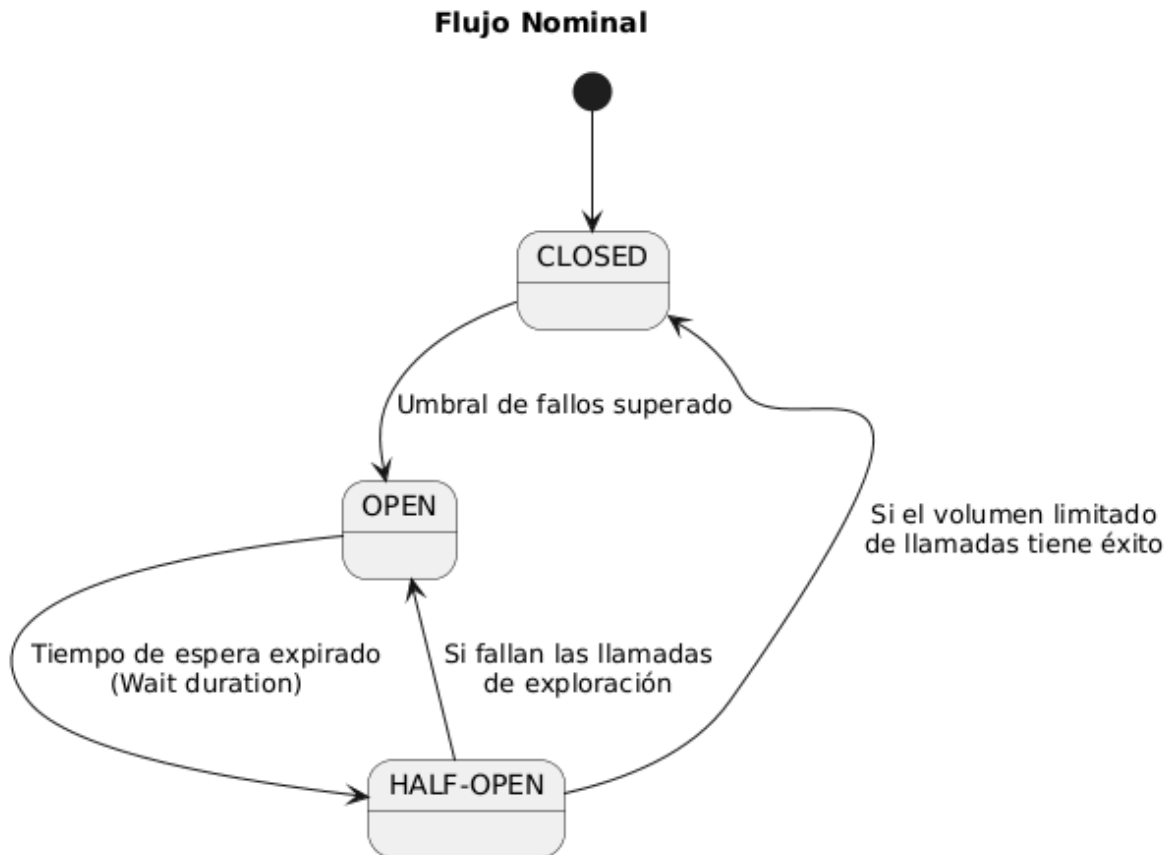
Para gestionar inteligentemente esta tolerancia, el Circuit Breaker opera bajo una máquina de estados finita gobernada por tres estados inmutables, requiriendo transiciones dinámicas para garantizar la auto-recuperación del sistema⁶.

CLOSED (Cerrado): El estado operacional estándar. El disyuntor permite que todas las peticiones fluyan hacia el servicio destino. De manera pasiva, registra el tiempo de ejecución y el resultado (éxito o excepción) de cada invocación en una ventana de observación temporal o cuantitativa. Mientras la tasa de fallos se mantenga por debajo del límite seguro estipulado, el estado no muta³⁰.

OPEN (Abierto): Cuando los algoritmos de monitoreo detectan que la acumulación de errores o respuestas lentas supera el umbral crítico, el mecanismo acciona la apertura del circuito. En este estado, se prohíbe terminantemente el paso de cualquier solicitud hacia el recurso comprometido. En su lugar, el interceptor aborta la operación instantáneamente, arrojando una excepción al código cliente o, preferiblemente, ejecutando una ruta de contingencia conocida como *Fallback* (por ejemplo, devolver información de una memoria caché secundaria o un valor por defecto). Esto libera de presión operativa (load shedding) al servicio agonizante, otorgándole el respiro computacional necesario para reiniciar procesos o purgar colas colapsadas⁶.

HALF-OPEN (Medio Abierto): El patrón no requiere intervención manual humana para rehabilitar el tráfico. Al ingresar al estado OPEN, el disyuntor arranca un temporizador. Una vez expirado este tiempo de espera, el circuito transita automáticamente a HALF-OPEN. Este es un estado exploratorio o de sondeo. Permite que un número estrictamente limitado y controlado de peticiones atraviese el filtro hacia el servicio destino para evaluar empíricamente si este se ha

recuperado. Si estas peticiones de prueba resultan en nuevos fallos o latencias inaceptables, el circuito asume que el sistema subyacente sigue comprometido y transita inmediatamente de regreso al estado OPEN, reiniciando el temporizador. Por el contrario, si las peticiones de prueba culminan con éxito, el disyuntor concluye que el incidente fue superado, transiciona a CLOSED y normaliza el caudal de tráfico total⁶.



Configuración

La calibración de los parámetros del Circuit Breaker representa un delicado equilibrio matemático. Un ajuste excesivamente celoso desencadenará cortes de servicio ante parpadeos inofensivos de la red (falsos positivos), mientras que una configuración permisiva permitirá la propagación de fallos en cascada (falsos negativos). Los parámetros de referencia operan con la siguiente semántica estructural²⁶:

- slidingWindowSize = 10 (Tamaño de la ventana deslizante):** Dictamina el volumen del historial que el circuito almacena para computar estadísticas. Existen ventanas basadas en recuento (count-based, analizando las últimas N peticiones) y basadas en tiempo (time-based, evaluando peticiones de los últimos N segundos). Un tamaño de 10 peticiones significa que cada nuevo llamado expulsa del registro al más antiguo en una estructura de búfer circular. Valores pequeños provocan alta volatilidad estadística; un solo fallo eleva la tasa de error al 10%. Ventanas inmensas, en contraste, hacen al

circuito letárgico, obligando al sistema a absorber severos daños antes de protegerse³⁰.

- **failureRateThreshold = 50% (Umbral de tasa de fallos):** Porcentaje estadístico de operaciones anómalas requerido dentro de la ventana para disparar el estado OPEN. En este escenario, si de las últimas 10 peticiones evaluadas, 5 o más concluyen en excepciones de red, el disyuntor se activa. Un umbral de 10% es excesivamente frágil en arquitecturas modernas con alta fluctuación de latencia. Un umbral de 50% asume una tolerancia moderada, asumiendo que el servicio está definitivamente comprometido si falla la mitad de las transacciones²⁸.
- **slowCallRateThreshold (Umbral de tasa de llamadas lentas):** Aislado del fallo total, este parámetro previene el estrangulamiento de hilos. Cataloga como lentas a aquellas peticiones que, aunque logran retornar una respuesta válida, exceden un umbral de tiempo (timeout) considerable, forzando la apertura si la degradación temporal supera el porcentaje acordado³¹.
- **waitDurationInOpenState = 10s (Duración de espera):** El periodo ininterrumpido en el que el circuito rechaza absolutamente el tráfico antes de sondear la recuperación. Configurarlos en 10 segundos asume que la interrupción típica en la dependencia obedece a reinicios rápidos de pods o recálculos efímeros de base de datos. Si el servicio de terceros típicamente demora cinco minutos en recuperarse de una sobrecarga, 10 segundos es un valor patológicamente bajo que causará "flapping" incesante del estado entre OPEN y HALF-OPEN, enviando ruido estéril a la red²⁶.
- **permittedNumberOfCallsInHalfOpenState = 3 (Peticiones en medio-abierto):** El volumen de peticiones reales permitidas como sonda para validar la curación del servicio dependiente. Un número de 3 permite recolectar suficiente evidencia de estabilidad para evitar transicionar a CLOSED tras una única respuesta afortunada (golpe de suerte estadístico), garantizando así una rehabilitación responsable²⁸.

Patrones complementarios

El Circuit Breaker exhibe todo su potencial cuando se orquesta sinérgicamente junto con otros patrones de resiliencia táctica (resilience engineering) y control de tráfico²⁶:

- **Retry (Reintento):** Diseñado para sobreponerse a errores de comunicación transitorios intentando cíclicamente reejecutar una operación. La yuxtaposición algorítmica de Retry más Circuit Breaker es excepcionalmente peligrosa si se modela mal. Si un cliente insiste repetitivamente en llamar a un servicio que está al límite de su capacidad, el patrón Retry multiplicará artificialmente el tráfico y propinará un ataque por denegación de servicio (DDoS) involuntario. El Circuit Breaker se interpone a esta conducta protegiendo la red, obligando al Retry a fracasar instantáneamente sin añadir carga real a la red, hasta que la tormenta pase²⁰.
- **Timeout (Tiempo de Espera):** Dictamina el límite de tolerancia antes de que un hilo claudique esperando respuesta. Provee el flujo de datos primario hacia el Circuit Breaker; un enjambre de timeouts nutre el umbral de errores o latencia que activa la apertura⁷.
- **Bulkhead (Mamparo):** Inspirado en los compartimientos estancos de los buques. Segmenta los recursos (hilos, memoria) por dominio de dependencia. Evita que la

congestión generada por las interacciones defectuosas con un servicio agote los hilos de los demás dominios no afectados. Conjuntamente con el Circuit Breaker, restringe el radio de explosión (blast radius) transversal².

- **Fallback:** Otorga la capacidad de proveer gracia en la degradación (graceful degradation). En lugar de propagar un crudo error 500 al cliente cuando el Circuit Breaker interrumpe el paso, el método Fallback ofrece una ruta secundaria, como despachar un mensaje encolado, entregar un valor de caché, o retornar una advertencia contextual, manteniendo la ilusión de operatividad en la capa frontal⁶.

Saga vs Circuit Breaker

Es imperativo discernir que Saga y Circuit Breaker habitan dimensiones operativas ortogonales; no son patrones sustitutos ni competidores, abordan patologías sistémicas radicalmente distintas en las arquitecturas modernas.

Aspecto	Saga Pattern	Circuit Breaker Pattern
Problema principal	La carencia transaccional distribuida y el secuestro de recursos por bloqueos persistentes.	Intermitencias de red foráneas y el agotamiento estructural del hardware (inanición de hilos).
Objetivo	Preservar inflexiblemente la Consistencia de Datos de negocio (Consistencia Eventual).	Blindar la Disponibilidad del ecosistema e instituir Tolerancia a Fallos ante caídas.
Tipo de problema	Lógica funcional y validez del dominio semántico empresarial (Business Logic).	Congestión de infraestructura, latencias sistémicas y de capa de transporte (Operational).
Consistencia	Reestructura el esquema de datos temporalmente hasta converger globalmente.	Acepta la pérdida momentánea de comunicación; prefiere la caída limpia a la inconsistencia fatal.
Fallos	Resuelve rechazos comerciales legítimos (e.g., saldo insuficiente, stock agotado).	Mitiga degradaciones del entorno (e.g., timeouts, puertos cerrados, lentitud de la CPU local).

Comunicación	Predominantemente orientada a Eventos Asíncronos a través de brokers o repositorios.	Predominantemente aplicable sobre comunicaciones Sincrónicas RPC o HTTP entre nodos.
Compensaciones	Impulsa contra-medidas comerciales explícitas persistiéndolas en bases de datos.	Simplemente detiene la transmisión hacia los linderos, bloquea y opcionalmente retorna fallbacks.
Estados	Máquina de estado orientada a dominios de negocio corporativos (Iniciado, Fallido, Compensando).	Máquina de estados enfocada a viabilidad transitoria y topología de red (Cerrado, Abierto).
Recuperación	Regresa lógicamente la integridad económica reajustando todo el sistema mediante acciones.	Concede una pausa curativa al proveedor dependiente negando el tráfico tóxico temporal.
Aplicación	Ejerce control macrológico de flujos sobre toda la topografía transaccional distribuida.	Microgestión local; envuelve invocaciones de punto a punto en fronteras de red específicas.

Integración conceptual

La simbiosis entre ambos patrones conforma la resiliencia definitiva de la arquitectura en la nube. Supóngase una infraestructura dominada por un orquestador de Saga que coordina transacciones diseminando comandos vía gRPC o HTTP hacia dependencias esparcidas. El patrón Circuit Breaker asume el mandato de resguardar el canal de comunicación. Si el servicio de inventario colapsa intempestivamente, las llamadas de red del orquestador experimentarán severos timeouts. Frente a este panorama, el Circuit Breaker envuelve el cliente HTTP del orquestador; excedida la tolerancia, quiebra el circuito. Todo nuevo comando hacia el inventario es rechazado localmente en microsegundos (Fail-fast), garantizando la inmunidad de la capacidad computacional y los hilos del orquestador⁷.

Confrontada al violento y deliberado fallo arrojado por el Circuit Breaker, la máquina de estados

del patrón Saga despliega su pericia de dominio. Evaluando la interrupción, la Saga comprende que se enfrenta a una catástrofe ajena a la lógica de negocio; no deduce que falta inventario, comprende que el proveedor es innacesible. La Saga puede resolver entonces aplazar la operación mediante reintentos retardados (apoyándose en una cola tolerante) asumiendo que el Circuit Breaker sanará eventualmente en estado HALF-OPEN. Alternativamente, dictaminada la caducidad operativa, la Saga puede rendirse, asumiendo el fallo global y emitiendo soberanamente los comandos de transacciones compensatorias para restituir la rectitud financiera en el sistema. El Circuit Breaker apantalla las vías comunicativas de la infraestructura; la Saga ejerce la potestad y la validación incuestionable del modelo de datos resultante³.

Caso de e-commerce

El análisis del escenario transaccional provisto en la práctica (Order Service → Payment Service → Inventory Service → Shipping Service) permite elucidar pragmáticamente esta amalgama.

Flujo exitoso:

1. **Crear orden:** El usuario interactúa remotamente con el *Order Service*. El sistema registra localmente la orden bajo el estado PENDING (Transacción local ACID).
2. **Cobrar cliente:** Se instruye al *Payment Service*. Este verifica su propia base relacional, procesa los pormenores financieros exógenos, disminuye los fondos por 100 USD (Transacción local ACID). Confirma el éxito.
3. **Descontar stock:** El *Inventory Service* verifica las exigencias contra su tabla inmutable de existencias; hay disponibilidad material. Amortiza las unidades, actualiza sus tablas (Transacción local ACID).
4. **Programar envío:** El *Shipping Service* pacta los esquemas de movilización transitoria, genera una guía y confirma (Transacción local ACID). El *Order Service* eleva la jerarquía del proceso al estado íntegro APPROVED.

Flujo con fallo de inventario:

1. **Crear orden:** Transacción persistida localmente sin fricción (PENDING).
2. **Cobrar cliente:** Transacción de merma confirmada localmente. Los fondos están formalmente arrebatados de la tutela del cliente.
3. **Falla inventario:** Invocado el dominio logístico subyacente, el *Inventory Service* inspecciona el catálogo, evidenciando un agotamiento de stock por carreras de concurrencia inoportunas. La transacción local aborta y arroja fallos semánticos formales.
4. **Revertir pago:** Alertada de la calamidad irresoluble, la Saga entra en modo contención. Emite directrices correctivas hacia el *Payment Service*. Este despliega la operación compensatoria predefinida: un reembolso de 100 USD, inyectando un nuevo registro de rectificación en sus tablas (Transacción local ACID compensatoria).
5. **Cancelar orden:** Corroborada la reparación monetaria, la orden es mutada desde PENDING a CANCELLED (Transacción local ACID final).

En ninguna faceta se apeló a un coordinador transaccional central de bloqueo para congelar las filas durante las evaluaciones lógicas dispares, garantizando una fluidez prístina y escalabilidad horizontal inalienable.

Escenarios de fallo

Un examen de resiliencia requiere diagnosticar cómo el sistema distribuido y sus patrones arquitectónicos reaccionan ante un espectro de patologías destructivas:

- **Caso 1: Payment falla (Fallo Lógico).** Ocurre un rechazo por fondos insuficientes. La Saga interviene de inmediato y ordena la cancelación de la orden. Es una reversión primaria sin afectación monetaria ni exposición de daño.
- **Caso 2: Inventory falla (Fallo Lógico).** Una sobreventa deja las bodegas carentes. Interviene la Saga ordenando explícitamente el reembolso hacia Payment Service y cancelando la orden primigenia. Existe el riesgo de confusión de los usuarios finales por lecturas sucias durante el hiato compensatorio donde el dinero aparece efímeramente incautado⁴.
- **Caso 3: Shipping falla (Fallo Transitorio).** La API del transportista niega la respuesta. Dado que los pasos peliagudos (pago e inventario) fueron sorteados exitosamente (transacción pivote superada), la Saga no debe compensar. Interviene el patrón Retry en conjunción con políticas de asincronía (Eventual Consistency), perseverando interminablemente hasta consolidar el despacho logístico⁴.
- **Caso 4: Un servicio deja de responder (Fallo de Red).** El Inventory Service desaparece de la topología causando un cúmulo de timeouts en sus requirientes. Interviene el patrón Circuit Breaker; tras exceder la `failureRateThreshold`, seccionará el circuito (OPEN). Las invocaciones se repudian en nanosegundos protegiendo los hilos del consumidor; un Fallback podría canalizar la decisión a una cola de retrasos o fallar rápido instruyendo a la Saga a declinar la orden comercialmente.
- **Caso 5: Un servicio responde lentamente (Degradación).** El Payment Service experimenta una carga inusitada procesando respuestas en 20 segundos. Interviene el Circuit Breaker fundamentado en la `slowCallRateThreshold`. Transiciona a OPEN inhibiendo que el sistema despilfarre todos sus hilos esperando resolución. Mitiga efectivamente la avalancha progresiva y paralización en cascada del clúster³¹.
- **Caso 6: Una compensación falla (Fallo Crítico).** Al tratar de reembolsar en la fase compensatoria, el Payment Service emite alertas HTTP 503. Interviene imperativamente el Idempotency Pattern combinado con reintentos perpetuos. La Saga persiste la necesidad compensatoria y la invoca recursivamente hasta acatarla. Es el mayor riesgo teórico, ya que la rendición aquí ocasionaría inconsistencias financieras perennes para la corporación²⁰.
- **Caso 7: Se reciben eventos duplicados (Asincronía).** Provocado por un parpadeo de red que impulsó entregas redundantes, Payment Service recibe múltiples mandatos para cobrar la misma factura. Interviene la Idempotencia dictaminada obligatoriamente sobre el consumidor. Usando la firma transaccional original, el servicio intercepta el dolo, ignora operativamente las repeticiones y simula un retorno exitoso resguardando la cuenta del usuario²⁰.
- **Caso 8: Un evento llega fuera de orden (Inconsistencia de Entregas).** Por retardos en el bróker, Order Service ingiere el edicto `OrderCancelled` antes del originario `OrderCreated`. Intervienen algoritmos de control de concurrencia optimista y gestión del

estado evolutivo. Order Service rechaza transmutar estados ilusorios, persiste el mandato temporalmente o lo supedita al evento matriz eludiendo la prevaricación de base de datos⁵.

Estrategia de pruebas

Demostrar científicamente la probidad arquitectónica de estos patrones frente a una simulación académica (PoC) requiere un marco metodológico automatizable, orientado no sólo a los casos asertivos (happy paths), sino fundamentado en tácticas de ingeniería del caos (chaos engineering) para verificar las propiedades resilientes³³.

Para evidenciar la solidez del patrón Saga:

- **Flujo completamente exitoso:** Las métricas de prueba deben cerciorar la inserción transaccional atómica e impecable sobre los esquemas locales del T1 al T4.
- **Fallo de Payment / Inventory:** Deberán introducirse datos anómalos o *mocks* deliberados desencadenantes de contingencias comerciales para certificar innegablemente el paso de la orden hacia las transacciones compensatorias oportunas (creación y verificación de registros de reembolso y anulación).
- **Fallo de Shipping:** Testificar que, post-fase pivote, un fracaso logístico inyectado no estimula compensaciones letales, sino colas de reintentos subrogados.
- **Ejecución y Verificación final de compensaciones:** Provocar caídas en un middleware justamente antes de la confirmación resarcitoria. Demostrar empíricamente cómo las estructuras retoman la carga y solidifican el estado integral (consistencia eventual garantizada) apenas los enlaces sanen.

Para comprobar la funcionalidad del Circuit Breaker:

- **Estado CLOSED:** Efectuar sondeos masivos de baja latencia; comprobar el fluir irrestricto.
- **Generación de errores y Superación del Threshold:** Introducir intencionalmente respuestas HTTP 500 y Timeouts ficticios que corrompan flagrantemente el contador de la ventana temporal y empujen al disyuntor por sobre la barrera configurada (ej. > 50%).
- **Estado OPEN y Fallback:** Atestiguar mediante las latencias métricas cómo el subsecuente caudal fracasa estrepitosamente en escala de microsegundos (Fail-Fast sin alcanzar el transporte de red) acudiendo llanamente a los mecanismos predefinidos de respuesta (Fallback)²⁶.
- **Espera y Estado HALF-OPEN:** Congelar las peticiones por el rango estricto estipulado en `waitDurationInOpenState`. Reiniciar levemente los sondeos. Demostrar la apertura controlada (sonda) con un flujo raquíptico permitiendo examinar cómo, si se remueve el error inyectado, el disyuntor recobra el temple cerrado tras los ensayos definidos, o en su defecto recae al precipicio aislacionista si el caos perdura²⁸.

Qué debe observarse en las pruebas

Las pruebas serán estériles si carecen de una política de observabilidad. Se deben recopilar evidencias analíticas cristalinas, descartando las evaluaciones dicotómicas "funciona/no funciona".

- **Para los Estados de Saga y Transacciones:** Se requiere evidenciar los ID de

Correlación (Trace IDs) rebotando ininterrumpidamente. Probar el registro temporal auditable (event logs / outbox tables) que exponga textualmente los comandos generados, consumidos, y el historial irrefutable de compensaciones registradas inalterables en las bases de datos de los participantes²³.

- **Para Estados del Circuit Breaker:** Presentar la gráfica o los logs explícitos arrojados por el interceptor local (e.g., WARN: CircuitBreaker 'paymentCB' changed state from CLOSED to OPEN due to 60% failure rate), junto a las métricas documentando la contención drástica en el consumo de los *threads*²⁶.

Ventajas y Desventajas

Un análisis intelectualmente honrado exige diseccionar críticamente las capacidades y penalidades implícitas introducidas en la asimilación arquitectural.

Saga Pattern:

- **Ventajas:** Fomenta una hiper-escalabilidad envidiable al pulverizar la perniciosa centralización de bloqueos (locks) de las bases de datos transaccionales 2PC⁴. Brinda libertad de innovación técnica, avalando la dispersión heterogénea y el desacoplamiento rotundo del ecosistema corporativo que opera orquestadamente sosteniendo la consistencia eventual y la solidez semántica de largo alcance.
- **Desventajas:** Detona irreparablemente la complejidad algorítmica. Engendrar compensaciones irrompibles con exigencias de idempotencia es gravoso y retardador²⁰. Promueve la opacidad operativa si se decanta por coreografías crudas. Abre el sistema a inconsistencias relacionales subrepticias originando pérdida de actualizaciones ante la inexistencia de aislamiento total (isolation anomalies)⁴. Su control de errores transmutado de sintáctico a comercial eleva el riesgo operativo a proporciones masivas en caso de desarrollos defectuosos.

Circuit Breaker Pattern:

- **Ventajas:** Baluarte crucial de resiliencia (resilience engineering) e inmunización perimetral de red. Impide categóricamente el descalabro sistémico masivo abortando caídas en cascada. Descarga a las infraestructuras degradadas auxiliándolas en su propia regeneración. Otorga contención matemática predecible⁶.
- **Desventajas:** Reclama una calibración paramétrica minuciosa y científica dependiente absolutamente del caso subyacente. Falsos positivos seccionarán prematuramente conexiones idóneas provocando frustraciones; una indulgencia superlativa arruinará el sistema soslayando la asfixia. Sus encubrimientos mediante *Fallbacks* sofisticados pueden camuflar problemas de infraestructura medulares y postergar alarmas sistémicas reales de degradación paulatina.

Casos donde no conviene utilizar cada patrón

La ortodoxia ingenieril dicta que los patrones arquitectónicos no son utopías resolutivas unívocas, sino sacrificios contextuales aplicables con pragmatismo dictaminado por los principios CAP y PACELC.

Saga es contraproducente cuando:

- La aplicación admite un encapsulamiento tradicional como Monolito Modular adosado a

una única base de datos SQL donde las garantías rígidas transaccionales (ACID locales) resultan inigualables en sencillez, protección e indemnidad¹.

- Dominios de exigencia milimétrica requiriendo consistencia linealizable (linearizability). Casos donde se intercambian derivados financieros de alta frecuencia exigiendo que cada cliente experimente simultáneamente el mismo valor estricto; en esos entornos la consistencia eventual con ventanas de lecturas sucias y compensaciones lentas es ilegal, técnica y procesalmente⁵.
- Circuitos algorítmicos colmados de dependencias cíclicas en las interconexiones que auspiciarían interbloqueos insuperables intentando transacciones retroactivas de compensación.

Circuit Breaker es superfluo cuando:

- Rige en enclaves de ejecución local monolíticos, librerías de componentes nativos o llamadas entre procesos internos (IPC), donde abulta el sobrecoste computacional previniendo fallos de red mitológicos inexistentes.
- Suplementa a integraciones desacopladas regidas radicalmente por colas de mensajería (Message Brokers) estandartes de flujos "Dispara y Olvida" (Fire and Forget). Si el cliente arroja un evento asíncrono y desaparece de la topografía, no adolece de amenazas latentes por cuellos de botella del lado prestatario, inutilizando la prevención del Circuit Breaker⁷.

Patrones relacionados

Ambos patrones permean un tejido mayor, exigiendo andamiajes satelitales para su correcto sustento sistémico en las nubes.

- **Transactional Outbox:** Imperioso aliado de la Saga. Intercepta la dualidad peligrosa de comprometer bases de datos y remitir correos al Bróker de red. Apunta el mandato en las tablas relacionales garantizando simetría y posterga la transmisión asíncrona de las señales, salvando la coherencia en fallos²².
- **Event Sourcing:** Almacena los estados no mediante celdas estáticas actuales sino a modo de registro histórico (log) perenne de incidencias. Ennoblecía radicalmente los reintegros transaccionales.
- **Dead Letter Queue (DLQ):** Retén purgatorio esencial para la resolución de reintentos frustrados continuos en Sagas. Alza las barreras ante fallos indómitos sin sepultar la salud del procesador primario y las retiene para maniobras operativas diferidas²².
- **Idempotency:** Cláusula dogmática para asimilar y procesar requerimientos duplicados gestados en resiliencia o mermas transaccionales; previene afectaciones indeseables gemelas²⁰.
- **CQRS (Command Query Responsibility Segregation):** Discrimina topográficamente las rutinas operativas (comandos) y visiones retrospectivas (lecturas). Ampara visiones prístinas previendo que Sagas en vilo irradian reportes enlodados de datos no consolidados y protegiendo al orquestador de colapsos interrogativos masivos⁵.
- **Distributed Tracing (Trazabilidad Distribuida):** Fundamental para correlacionar la explosión de asincronía en las Sagas coreografiadas y depurar errores silenciosos.

Conclusiones

La disección rigurosa de los marcos operativos y la teoría de redes destapa resoluciones dogmáticas imprescindibles frente a la futura construcción analítica del proyecto:

Saga Pattern: Las demostraciones fundamentadas exponen a la Saga no como un parangón exacto de las resoluciones ACID vetustas, sino como la resignificación filosófica y estructural de la viabilidad de la base de datos empresarial fragmentada⁵. Abandona el confort dictatorial y bloqueante del protocolo 2PC y asimila las cicatrices de la Consistencia Eventual en sistemas distribuidos. Requiere, insustituiblemente, la adopción rigurosa de arquitecturas compensatorias robustecidas por la idempotencia y el asilamiento de sus fases pivote⁴. Su adopción asume una enorme carga cognitiva y de programación compensatoria sobre las filas operativas, intercambiando astucia lógica por fluidez, escalabilidad y libertad organizativa extrema en ecosistemas masivos¹⁷.

Circuit Breaker Pattern: Su validación analítica denota que este arquetipo no pretende sostener los dominios semánticos empresariales, sino blindar los tejidos conectores del agotamiento por fatiga térmica. Las evidencias teóricas amparan que las calamidades en cascada nacen del secuestro sigiloso de hilos en modo de letargo y latencias parasitarias⁷. Actuando mediante ventanas de observación precisas, su labor protectora interrumpe la avalancha del "desbordamiento de recursos" en fracciones métricas mediante respuestas perentorias (Fail Fast) resguardando celosamente a cada cliente de sus toxicidades enlazadas, al tiempo que facilita la regeneración de sus colapsadas dependencias perimetrales⁶.

Uso conjunto y Aplicación (E-commerce): El amalgamamiento en escenarios integrados ratifica la interdependencia simbiótica³. Aplicado pragmáticamente sobre las disrupciones de un servicio de despacho comercial y pagos, el Circuit Breaker repelerá asfixias inmediatas cercenando estancamientos logísticos. Tras la innegable obstrucción originada, la arquitectura Saga asimilará el diagnóstico de contingencia provisto para reestructurar la información alterada del usuario a un cimiento firme mediante comandos compensatorios indemnizatorios. Mientras el interceptor en red mitiga daños infraestructurales, el orquestador de eventos asegura la pureza moral y procedimental de las cuentas comerciales de la empresa. Este cimiento teórico blindará conceptualmente cualquier incursión tecnológica posterior, garantizando que el diseño obedece al pragmatismo distribuido comprobado.

Fuentes citadas

1. Database Series : CAP Theorem - Medium,
<https://medium.com/@abhi.strike/database-series-cap-theorem-14e1741e78a5>
2. 50 Core System Design Concepts - MaxiDica,
<https://maxidica.com.br/2026/03/04/50-core-system-design-concepts/>
3. From CAP to PACELC: Designing Smarter Distributed Systems,
<https://medium.com/@techboosting/from-cap-to-pacelc-designing-smarter-distributed-systems-63e237c2a6da>
4. Distributed Transactions: Sagas - Rafee Mohamed,
<https://rafeemohamed.com/blogs/distributed-transactions-sagas/>

5. Distributed Transactions: Two-Phase Commit, Sagas, and, <https://distributedsystemauthority.com/distributed-transactions/>
6. Efficient Fault Tolerance with Circuit Breaker Pattern - Aerospike, <https://aerospike.com/blog/circuit-breaker-pattern/>
7. Circuit Breaker - Martin Fowler, <https://martinfowler.com/bliki/CircuitBreaker.html>
8. ANALYSIS OF DISTRIBUTED TRANSACTION PATTERNS WITHIN, <https://digikogu.taltech.ee/et/Download/1ac99433-980f-43d5-87fe-852e45c6d2dc>
9. Data Management in Microservices: State of the Practice ... - arXiv, <https://arxiv.org/html/2103.00170v2>
10. CAP Theorem | System Design - AlgoMaster.io, <https://algomaster.io/learn/system-design/cap-theorem>
11. Consensus on Transaction Commit - s2.SMU, <https://s2.smu.edu/~mhd/8330f11/p133-gray.pdf>
12. Eight Transaction Papers by Jim Gray | alphaXiv, <https://www.alphaxiv.org/abs/2310.04601>
13. A New Presumed Commit Optimization for Two Phase Commit, <https://15799.courses.cs.cmu.edu/fall2013/static/papers/lampson-2pc.pdf>
14. Consensus on Transaction Commit - arXiv, <https://arxiv.org/pdf/cs/0408036>
15. Two-Phase Commit, and the window nothing closes - Logical Art, <https://logicalart.us/nibbles/two-phase-commit/>
16. Atomic Commit in Distributed Database Systems, <https://www.ijert.org/research/atomic-commit-in-distributed-database-systems-the-approaches-of-blocking-and-non-blocking-protocols-IJERTV3IS100052.pdf>
17. (PDF) Design and Evaluation Of Event-Driven Architectures for, https://www.researchgate.net/publication/384018359_Design_and_Evaluation_Of_Event-Driven_Architectures_for_Transaction_Management_in_Microservices
18. Consistency Models in Distributed Microservices - Semantic Scholar, <https://pdfs.semanticscholar.org/07f5/1da164ed527816e9941622c22993c8f05ca1.pdf>
19. Understanding the Azure Saga Pattern | PDF | Database Transaction, <https://www.scribd.com/document/829667089/Saga-pattern-Azure-Design-Patterns-Microsoft-Learn>
20. Resilience patterns reconsidered: Sagas, Circuit Breakers, and, <https://www.saumilp.dev/blog/resilience-patterns-reconsidered/>
21. Dan Plyukhin, <https://dplyukhin.github.io/>
22. Distributed Transactions and why you should avoid them - Github-Gist, https://gist.github.com/rponte/9477858e619d8b986e17771c8be7827f?permalink_comment_id=4105335
23. Distributed Transactions and why you should avoid them - GitHub Gist, https://gist.github.com/rponte/9477858e619d8b986e17771c8be7827f?permalink_comment_id=5175508
24. Circuit breaker pattern - AWS Prescriptive Guidance, <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/circuit-breaker.html>
25. The Circuit Breaker Pattern | Aerospike Documentation,

- <https://aerospike.com/docs/develop/tutorials/circuit-breaker/>
26. blackblacktigerHQ/resilience-circuitbreaker: Distributed circuit, <https://github.com/blackblacktigerHQ/resilience-circuitbreaker>
 27. Circuit Breaker Pattern Stop Cascading Failure Fast - Medium, <https://medium.com/beyond-localhost/microservices-pattern-circuit-breaker-stop-cascading-failures-before-they-cascade-427f0f88b033>
 28. Circuit Breaker Pattern for Resilient Systems - DZone, <https://dzone.com/articles/circuit-breaker-pattern-resilient-systems>
 29. MCP Circuit Breaker: Preventing Cascading Failures in AI Tool Calls, <https://dev.to/neurolink/mcp-circuit-breaker-preventing-cascading-failures-in-ai-tool-calls-4bi4>
 30. CircuitBreaker - resilience4j - ReadMe, <https://resilience4j.readme.io/docs/circuitbreaker>
 31. A Circuit Breaker Story — How to save your valuable services from, <https://engineering.getmidas.com/a-circuit-breaker-story-how-to-save-your-valuable-services-from-burning-bf9064084ba4>
 32. Resiliency Pattern — CircuitBreaker | by Govind Yadav - Medium, <https://medium.com/@govindjadon/resiliency-pattern-circuitbreaker-4363bdbe581b>
 33. Distributed Systems Design Patterns: A Practitioner Reference, <https://distributedsystemauthority.com/distributed-systems-design-patterns/>
 34. Pattern: Transactional outbox - Microservices.io, <https://microservices.io/patterns/data/transactional-outbox.html>